TRABAJO PRÁCTICO SEMANA 2

Testing Práctico y Contratos

Tecnicatura Universitaria en Programación a Distancia – UTN

Apellido y nombre:	Figueroa, Ignacio – DNI 45.406.120
Comisión:	C4 - A26

Uso el mismo proyecto Tienda del TP de la semana 1 (Python + pytest). Las herramientas de dobles de prueba son de la librería estándar (unittest.mock).

1. Implementación del patrón AAA

Elegí la función aplicar_descuento, porque tiene varias reglas que se combinan:

src/tienda/precios.py

```
def aplicar_descuento(total, es_socio, cupon=None):
    """Aplica los descuentos.

    - Los socios tienen 10%.
    - El cupón PROMO15 da 15%, pero solo si la compra supera los $10.000.
    - Si corresponden los dos, se aplica solo el mayor (no se acumulan).
    """
    if total <= 0:
        return 0

    descuento = 0
    if es_socio:
        descuento = DESCUENTO_SOCIO
    if cupon == CUPON_VALIDO and total > MINIMO_CUPON:
        descuento = max(descuento, DESCUENTO_CUPON)

    return round(total * (1 - descuento), 2)
```

Cada test sigue las tres etapas, separadas con comentarios:

- **Arrange:** preparo los datos de entrada (el total, si es socio, el cupón).
- **Act:** ejecuto una sola acción, la función que estoy probando.
- **Assert:** verifico el resultado esperado.

Traté de que cada test pruebe una sola cosa y que el nombre diga qué se espera, así cuando falla se entiende el problema sin leer el código. Dos ejemplos de la suite:

tests/test_precios.py (fragmento)

```
"""Pruebas unitarias de precios, siguiendo el patrón AAA (Arrange, Act, Assert)."""

import pytest

from tienda.precios import aplicar_descuento, calcular_subtotal

# ---------- calcular_subtotal ----------


def test_subtotal_suma_precio_por_cantidad():
    # Arrange
    items = [{"precio": 1000, "cantidad": 2}, {"precio": 500, "cantidad": 3}]
```




```
    # Act
    resultado = calcular_subtotal(items)

    # Assert
    assert resultado == 3500


def test_socio_con_cupon_valido_se_queda_con_el_mayor():
    # Arrange
    total = 20000

    # Act
    resultado = aplicar_descuento(total, es_socio=True, cupon="PROMO15")

    # Assert: 15% y no 10% + 15%
    assert resultado == 17000
```

En los casos donde se espera una excepción, el Act y el Assert van juntos dentro de pytest.raises, porque la verificación ocurre en el momento de ejecutar. Resultado de la suite completa:

```
$ pytest -v tests/test_precios.py tests/test_pagos.py
tests/test_precios.py::test_subtotal_suma_precio_por_cantidad PASSED
tests/test_precios.py::test_subtotal_de_carrito_vacio_es_cero PASSED
tests/test_precios.py::test_subtotal_con_precio_negativo_lanza_error PASSED
tests/test_precios.py::test_total_cero_devuelve_cero PASSED
tests/test_precios.py::test_no_socio_sin_cupon_paga_precio_completo PASSED
tests/test_precios.py::test_socio_sin_cupon_tiene_10_por_ciento PASSED
tests/test_precios.py::test_cupon_valido_con_compra_mayor_al_minimo_da_15_por_ciento PASSED
tests/test_precios.py::test_cupon_valido_con_compra_menor_al_minimo_no_aplica PASSED
tests/test_precios.py::test_cupon_invalido_no_aplica PASSED
tests/test_precios.py::test_socio_con_cupon_valido_se_queda_con_el_mayor PASSED
tests/test_pagos.py::test_pago_aprobado_devuelve_id_de_transaccion PASSED
tests/test_pagos.py::test_pago_rechazado_lanza_excepcion PASSED
tests/test_pagos.py::test_cobrar_llama_a_la_api_una_vez_con_los_datos_correctos PASSED
tests/test_pagos.py::test_monto_invalido_no_llama_a_la_api PASSED
tests/test_pagos.py::test_devolver_pide_el_reembolso_de_la_transaccion PASSED
============================== 15 passed in 0.04s ==============================
```

2. Dobles de prueba: Stub y Mock (API externa de pagos)

La clase ProcesadorPagos usa una pasarela de pagos externa (por ejemplo, Mercado Pago). En los tests no conviene llamar a la API real, porque es lenta, puede estar caída, cobra plata de verdad y no deja provocar fácilmente casos como un rechazo. Por eso la pasarela se recibe por parámetro en el constructor y en los tests se reemplaza por un doble.

	Stub	Mock
Para qué sirve	Inyectar datos indirectos: le hace "decir" a la API lo que necesito (aprobado, rechazado, error).	Auditar la interacción: verificar que mi código llamó a la API como corresponde.
Qué verifico	El estado: qué devuelve o qué excepción lanza mi código con esa respuesta.	El comportamiento: cuántas veces se llamó, con qué parámetros y si no se debía llamar.
Cuándo usarlo	Cuando me importa qué hace mi código con la respuesta de la API.	Cuando la llamada en sí es lo importante, por ejemplo no cobrar dos veces o no cobrar si el monto es inválido.

	Stub	Mock
En el proyecto	PasarelaStub, una clase propia que devuelve una respuesta fija.	unittest.mock.Mock con assert_called_once_with y assert_not_called.

En un sistema de pagos, el **Stub** es necesario para probar cómo reacciona mi código ante cada respuesta de la API, por ejemplo qué pasa si rechaza la tarjeta por fondos insuficientes. Ese caso es muy difícil de provocar con la API real. El **Mock** es indispensable cuando el error estaría en la comunicación con la API y no en el resultado: si por un bug se llamara dos veces a cobrar(), el test de estado podría pasar igual (devuelve el id de transacción), pero al cliente se le cobraría dos veces. Solo un mock puede detectar eso. En resumen, con el stub controlo lo que entra a mi componente desde la dependencia, y con el mock controlo lo que mi componente le manda a la dependencia. En los dos casos el componente queda aislado de la API real.

tests/test_pagos.py

```
"""Pruebas de pagos con dobles de prueba (Stub y Mock) de la librería estándar."""

from unittest.mock import Mock

import pytest

from tienda.pagos import PagoRechazado, ProcesadorPagos


class PasarelaStub:
    """STUB: devuelve una respuesta fija. No verifica nada, solo inyecta datos."""

    def __init__(self, respuesta):
        self.respuesta = respuesta

    def cobrar(self, monto, token):
        return self.respuesta


def test_pago_aprobado_devuelve_id_de_transaccion():
    # Arrange: el stub simula que la API aprobó el pago
    stub = PasarelaStub({"estado": "aprobado", "id_transaccion": "TX-123"})
    procesador = ProcesadorPagos(stub)

    # Act
    resultado = procesador.cobrar(1500, "tok_visa")

    # Assert: verificamos el estado (lo que devuelve nuestro código)
    assert resultado == "TX-123"


def test_pago_rechazado_lanza_excepcion():
    # Arrange: el stub simula un rechazo, algo difícil de provocar con la API real
    stub = PasarelaStub({"estado": "rechazado", "motivo": "fondos insuficientes"})
    procesador = ProcesadorPagos(stub)

    # Act + Assert
    with pytest.raises(PagoRechazado, match="fondos insuficientes"):
        procesador.cobrar(1500, "tok_visa")
```

```
def test_cobrar_llama_a_la_api_una_vez_con_los_datos_correctos():
    # Arrange: MOCK, porque queremos auditar CÓMO se usa la API
    pasarela = Mock()
    pasarela.cobrar.return_value = {"estado": "aprobado", "id_transaccion": "TX-9"}
    procesador = ProcesadorPagos(pasarela)

    # Act
    procesador.cobrar(2500, "tok_master")

    # Assert: verificamos la interacción (que no cobre dos veces)
    pasarela.cobrar.assert_called_once_with(monto=2500, token="tok_master")


def test_monto_invalido_no_llama_a_la_api():
    # Arrange
    pasarela = Mock()
    procesador = ProcesadorPagos(pasarela)

    # Act
    with pytest.raises(ValueError):
        procesador.cobrar(0, "tok_visa")

    # Assert: nunca se tiene que haber llamado a la API
    pasarela.cobrar.assert_not_called()


def test_devolver_pide_el_reembolso_de_la_transaccion():
    # Arrange
    pasarela = Mock()
    procesador = ProcesadorPagos(pasarela)

    # Act
    procesador.devolver("TX-123")

    # Assert
    pasarela.reembolsar.assert_called_once_with(id_transaccion="TX-123")
```

3. Análisis de cobertura: ramas vs. sentencias

El segmento analizado es aplicar_descuento (mostrado en el punto 1). Tiene tres condiciones, y la última es compuesta: cupon == "PROMO15" and total > 10000.

La **cobertura de sentencias** solo mide si cada línea se ejecutó al menos una vez. La **cobertura de ramas** exige que cada if se haya evaluado por verdadero y por falso. El problema de la cobertura de sentencias es que un if sin else puede tener todas sus líneas ejecutadas sin que nunca se haya probado el caso en que la condición es falsa. Lo comprobé con el proyecto, corriendo solo 5 de los 10 tests:

```
$ pytest -k "<5 tests>" --cov  (solo cobertura de SENTENCIAS)
Name                     Stmts   Miss  Cover   Missing
src/tienda/precios.py       20      0   100%
5 passed, 5 deselected in 0.08s
```

```
$ pytest -k "<5 tests>" --cov  (con cobertura de RAMAS)
Name                     Stmts   Miss Branch BrPart  Cover   Missing
src/tienda/precios.py       20      0     10      2    93%   30->32, 32->35
5 passed, 5 deselected in 0.05s

$ pytest tests/test_precios.py --cov  (los 10 tests, RAMAS)
Name                     Stmts   Miss Branch BrPart  Cover   Missing
src/tienda/precios.py       20      0     10      0   100%
10 passed in 0.05s
```

Con los mismos 5 tests, la cobertura de sentencias marca **100%**, pero la de ramas marca **93%** y avisa que faltan dos caminos: 30->32 (el cliente **no** es socio) y 32->35 (el cupón **no** aplica). Es decir, nunca se probó qué pasa cuando no corresponde ningún descuento, y ahí podría haber un bug que la cobertura de sentencias no muestra. Por eso la cobertura de ramas es más rigurosa.

Casos de prueba para el 100% de cobertura de ramas

#	Entrada (total, socio, cupón)	total <= 0	es_socio	cupón válido y total > 10000	Resultado
1	0, sí, PROMO15	Verdadero	-	-	0
2	5000, no, ninguno	Falso	Falso	Falso	5000
3	5000, sí, ninguno	Falso	Verdadero	Falso	4500
4	20000, no, PROMO15	Falso	Falso	Verdadero	17000
5	8000, no, PROMO15	Falso	Falso	Falso (por el monto)	8000
6	20000, no, TRUCHO	Falso	Falso	Falso (por el cupón)	20000
7	20000, sí, PROMO15	Falso	Verdadero	Verdadero	17000

Con los casos 1 a 4 ya alcanza el 100% de ramas, porque cada if pasa por verdadero y por falso. Igual agregué el 5 y el 6 para que cada parte de la condición compuesta sea falsa por separado (cupón válido con compra chica, y compra grande con cupón inválido). Si no, un error en una de las dos partes podría pasar sin que ningún test lo detecte. El caso 7 prueba la regla de que los descuentos no se acumulan. Con la suite completa, el reporte da 100% de ramas.